

# Site Reliability Engineering (SRE) 系统可靠性工程

---

## SRE 的核心原则

虽然不同 SRE 团队在 workflows、优先级和日常运维方式上会有所不同，但所有 SRE 团队对于其所支持的服务，都承担一组基本职责，并遵循相同的核心原则。

一般来说，SRE 团队负责其服务的可用性、延迟、性能、效率、变更管理、监控、紧急响应以及容量规划。我们制定了一套规则和原则，用来规范 SRE 团队如何与其所处环境进行互动。这里的“环境”不仅包括生产环境，也包括产品开发团队、测试团队、用户等。

这些规则和工作方式帮助我们保持重点在工程工作上，而不是陷入纯粹的运维工作中。

以下内容将讨论 Google SRE 的几个核心原则。

---

## 保持对工程工作的长期关注

如前所述，Google 将 SRE 的运维工作时间限制在其总工作时间的 50% 以内。剩余时间应当用于发挥他们的编程能力，投入到项目型工程工作中。

在实际操作中，这需要持续监控 SRE 正在承担多少运维工作。如果运维工作过多，就要把超出的部分重新转回产品开发团队，例如：把 bug 和工单重新分配给开发经理，或者让开发人员重新加入 on-call 值班轮班等。

当运维负载重新下降到 50% 或以下时，这种转移才会结束。这也形成了一种有效的反馈机制，促使开发人员构建不需要大量人工干预的系统。

这种做法要想有效，需要整个组织——包括 SRE 和开发团队——都理解为什么存在这种“安全阀”机制，并共同支持这样一个目标：产品本身不应产生过多的运维负载，从而避免触发这种溢出机制。

当 SRE 专注于运维工作时，平均来说，在每个 8 到 12 小时的 on-call 值班班次中，SRE 最多应收到两个事件通知。这样的事件量可以让值班工程师有足够时间准确、快速地处理事件，清理问题，恢复正常服务，然后进行事故复盘。

如果每个 on-call 班次经常发生超过两个事件，问题就无法被充分调查，工程师也会因为负担过重而无法从事件中学习。告警疲劳的问题也不会随着规模扩大而自然改善。

相反，如果 on-call SRE 每个班次长期收到少于一个事件通知，那么让他们保持值班状态也是对其时间的浪费。

对于所有重大事故，都应该撰写事故复盘报告，无论这些事故是否触发了告警。事实上，那些没有触发告警的事故复盘可能更有价值，因为它们很可能揭示了明显的监控盲区。

事故调查应当详细说明发生了什么，找出事件的所有根本原因，并分配后续行动项，以修复问题，或改进下一次处理类似问题的方式。

Google 采用的是一种**无责事故复盘文化**。其目标不是追责或掩盖问题，而是暴露系统中的缺陷，并通过工程手段修复这些缺陷。

---

## 在不违反服务 SLO 的前提下追求最大变更速度

产品开发团队和 SRE 团队之间可以建立富有成效的合作关系，前提是消除双方目标之间的结构性冲突。

这种结构性冲突存在于两个目标之间：一方面是创新速度，另一方面是产品稳定性。正如前文所说，这种冲突往往会以间接方式表现出来。

在 SRE 中，我们把这种冲突明确摆到台面上，然后通过引入 **Error Budget（错误预算）** 来解决它。

错误预算这一概念来自一个观察：对于绝大多数系统而言，100% 并不是正确的可靠性目标。心脏起搏器和汽车防抱死制动系统等极少数场景可能是例外。

一般来说，对于任何软件服务或系统，100% 都不是合适的可靠性目标。原因是，用户通常无法感知一个系统是 100% 可用，还是 99.999% 可用。

在用户和服务之间还存在许多其他系统，例如用户的笔记本电脑、家庭 Wi-Fi、互联网服务提供商、电网等。这些系统整体上的可用性远低于 99.999%。

因此，99.999% 和 100% 之间的边际差异，往往会被其他不可用因素的噪声淹没。为了额外获得最后 0.001% 的可用性，需要投入巨大成本，但用户却未必能获得实际收益。

如果 100% 不是一个系统正确的可靠性目标，那么什么才是正确的可靠性目标呢？

这其实根本不是一个纯技术问题，而是一个**产品问题**。它需要考虑以下因素：

- 根据用户使用产品的方式，什么样的可用性水平会让用户满意？
- 如果用户对产品可用性不满意，他们有哪些替代选择？
- 在不同可用性水平下，用户对产品的使用行为会发生什么变化？

系统的可用性目标必须由业务或产品来确定。一旦这个目标确定下来，错误预算就是：

## 1 - 可用性目标

例如，一个服务的可用性是 99.99%，那么它的不可用性就是 0.01%。这个被允许的 0.01% 不可用时间，就是该服务的错误预算。

只要我们不超支，就可以把这个预算用在任何我们认为值得的地方。

那么，我们希望怎样使用错误预算呢？

开发团队希望发布功能，吸引新用户。理想情况下，我们会把错误预算花在快速发布所带来的风险上。这个基本前提描述了错误预算模型的核心思想。

一旦 SRE 活动被放进这个框架中，我们就可以通过一些策略来释放或保护错误预算，例如分阶段发布、1% 实验等，从而优化更快的产品发布速度。

错误预算的使用，解决了开发团队和 SRE 团队之间的激励冲突。

SRE 的目标不再是“零故障”。相反，SRE 和产品开发人员共同的目标是：合理使用错误预算，以获得最大的功能发布速度。

这一变化非常关键。

故障不再被简单地视为“坏事”。它是创新过程中预期会出现的一部分，是开发团队和 SRE 团队共同管理的现象，而不是需要恐惧的东西。